DO CONTEXT ENGINES HELP? RETRIEVAL QUALITY, DETERMINISM, AND AGENT UTILITY UNDER MATCHED BASELINES AND AUDITED EXPOSURE

Anonymous authors
Paper under double-blind review

ABSTRACT

Coding agents are increasingly wrapped in "context engines" that index a repository and return the files the model might need. We evaluate one open-source engine, Delphi, against lexical baselines, a matched ladder of conventional retrievers, and two hosted engines, under a protocol that records at the case level what each evaluation set could have influenced. Three findings temper our own earlier claims. *Exposure:* the largest partition we had called "untouched"—220 Agent Retrieval Bench cases—was re-drawn from a pool whose cases had all been scored in an earlier round and whose aggregate results gated which retrieval components shipped; we relabel it validation evidence and rest confirmatory claims on sets never scored before their single run: 62 SWE-bench Verified localization instances, 18 repository-disjoint commit-to-files cases, and a further 98 Verified instances drawn under a fresh salt. *Baselines:* Delphi beats BM25 and the official lexical ranker on every set, but against a conventional dense+BM25 hybrid built from the same embedding model, with Delphi's own cross-encoder, listwise reranker, and query expansion attached, the picture splits: on 160 pooled SWE-bench instances Delphi leads that stack by $+0.06$–0.08 on Recall@5, Recall@20, and budget-constrained yield with repository-cluster intervals excluding zero and ties on MRR ($+0.057$ $[-0.036, +0.114]$); on the 18-case independent final the conventional stack leads with intervals excluding zero. The reranking head, not anything specific to Delphi, accounts for most of the margin over lexical systems ($+0.3$ MRR versus $+0.06$). *Contracts:* routing every engine's retrieved evidence, including the hosted synthesis engine's own retrieved sources, through one frozen synthesis stage yields a four-way documentation tie ($0.625/0.617/0.583/0.575$) sitting 0.08–0.13 above a 0.492 no-retrieval floor; no engine-versus-engine interval excludes zero. On like-for-like retrieved-set stability the hosted synthesis engine is fully stable and Delphi is 98% stable; the "0%" we had reported for it measured generated text. A preregistered executable pilot on the same 62 SWE-bench instances (mini-SWE-agent, fixed budget, two trajectories per cell) resolves no seed effect: a Delphi seed changes verified repair by $+0.048$ $[-0.008, +0.113]$ over no seed and by -0.008 over a random-file control, and the two repeats disagree by more than the effects under study. We claim no state of the art. The contribution is the protocol, the negative and null results, and an exposure ledger that lets a reader check which number can bear which claim. Code, per-case artifacts, agent trajectories, and the ledger will be released.

1 INTRODUCTION

The pitch for a context engine is simple. Index the repo. Embed the chunks. When the agent asks a question, return the files that matter. The pitch for *not* building one is simpler: `rg` is deterministic, takes 30 milliseconds, and already found `src/_pytest/assertion/rewrite.py` when we asked where pytest rewrites asserts.

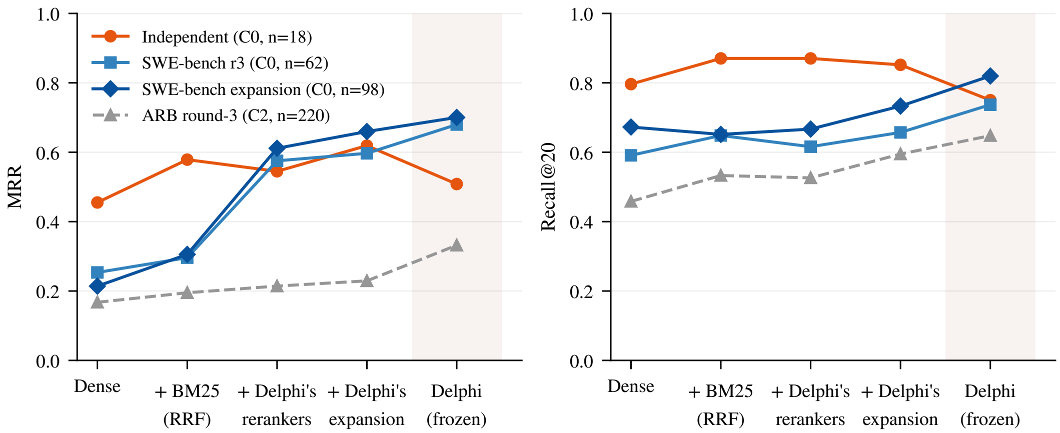

Figure 1: Where the margin comes from. Each conventional rung adds one Delphi component to the previous one over the identical corpus (§3); the last point is the frozen Delphi stack. On issue-style queries (SWE-bench) the reranking head moves MRR by roughly $+0.3$ and everything Delphi adds on top by $+0.04$–0.08; on the commit-to-files final Delphi finishes below the conventional stack. The dashed ARB line is the re-partitioned set on which Delphi's components were selected (class C2) and is shown for context only.

We built Delphi as a local-first retrieval stack—hybrid lexical/vector search, query expansion, a cross-encoder, a listwise rerank—and spent two evaluation rounds measuring it. The first round reported large wins over lexical baselines on a partition we called untouched. External review asked three questions we could not answer from the draft: what information from the earlier round was available while the shipped system was chosen; how much of the margin survives against a conventional dense-plus-lexical hybrid that uses the same reranker; and whether any of it changes what a coding agent actually repairs. This paper is the answer, and much of it is unflattering.

1. **RQ1 (utility).** Holding the agent, its tools, and its budget fixed, does a retrieval seed change what it submits—and does it change verified repair?
2. **RQ2 (ranking).** Against lexical baselines, against a matched conventional hybrid, and against hosted engines, where does Delphi lead, tie, or trail, on cases that no development decision could see?
3. **RQ3 (determinism).** Where does run-to-run variance come from, and when the same quantity is measured for every engine, who is actually stable?

Short answers. **RQ2:** Delphi leads lexical baselines everywhere; against a conventional ladder built from its own parts it leads modestly on 160 SWE-bench issue queries (three of four metrics excluding zero) and trails on the 18-case commit-to-files final (Figure 1, §4). **RQ3:** the variance was ours and is fixed in-process, but the like-for-like comparison shows the hosted synthesis engine's retrieval is at least as stable as ours (§5). **RQ1:** seeds move a closed-tool agent's file selection, replicating the benchmark's own intervention; on executable repair no seed effect is resolved at n=62 with two trajectories, and a random-file block does as well as Delphi's (§7).

2 Related work

SWE-bench established executable repository-level issue resolution (Jimenez et al., 2024), while SWE-agent and Agentless expose repository navigation and localization as explicit stages (Yang et al., 2024; Xia et al., 2024). LocAgent studies a graph-guided localization agent and reports that better localization can improve multi-attempt repair (Chen et al., 2025). Repository-level code completion provides complementary evidence that cross-file retrieval matters once the edit location is known: RepoCoder alternates retrieval and generation (Zhang et al., 2023); CodeRAG adds multi-path retrieval and preference-aligned reranking (Zhang et al., 2025). Dense retrieval, late interaction, and hypothetical-document expansion motivate the components in practical context engines

(Karpukhin et al., 2020; Khattab & Zaharia, 2020; Gao et al., 2023), while BM25 remains a strong exact-term baseline for code (Robertson & Zaragoza, 2009).

The closest work measures context acquisition directly. ContextBench provides human-verified file, block, and line contexts for issue-resolution tasks and scores agent trajectories (Li et al., 2026). CORE-Bench scales requirement-driven retrieval across code understanding, edit localization, and broader context (Zhang et al., 2026a). SWE-Explore derives audited line-level targets from successful trajectories (Zhang et al., 2026b). Agent Retrieval Bench (ARB) contributes workflow-derived next-context targets, base-commit hygiene, budget-aware file metrics, open-embedding and RepoMap baselines, and a closed-tool seed intervention with no-seed, random, retrieval-derived, and oracle arms (Qin & Xie, 2026). Our closed-tool experiment (§7) is a replication of that intervention with an additional seed, not a new design. What we add to this line is narrower: an exposure ledger that classifies every case by what could have influenced it, a component-matched baseline ladder, matched output contracts for hosted engines, like-for-like determinism, and a preregistered executable pilot on the same tasks as the ranking evaluation.

3 PROTOCOL

Exposure classes. Every evaluation set is assigned a class in a case-level ledger (Appendix C, Figure 6). **C0**: never scored by any system before its single confirmatory run, drawn from a pool no development decision could see. **C1**: development. **C2**: re-partitioned from a pool whose cases were all scored in an earlier round, where that round's aggregates influenced the shipped system. The ARB round-3 "final" is C2: all 427 ARB v2 cases were scored in July 2026; the July held-out aggregates were the acceptance test for query expansion, listwise reranking, the cross-encoder blend, reciprocal-rank fusion, and the path-affinity branch (Appendix E), and the July per-workflow rates motivated the quoted-path demotion fix. Round 3 then drew a new $25/75$ split from the same pool, excluded 50 legacy-exposed IDs, and queried the 220-case partition once. Re-drawing does not unexpose a case. Round-2 per-case outputs are not retained, so per-case tuning can neither be ruled in nor out from the archive; we therefore treat the partition as validation evidence throughout and make no confirmatory claim on it. The C0 sets are: 62 stratified SWE-bench Verified localization instances (12 repositories; patch-marker leakage audit $0/62$); an 18-case commit-to-files final over a repository-disjoint corpus locked before any scoring; and 100 further Verified instances drawn on 2026-09-09 under a fresh salt from the 438 instances no round had used (98 after two engine-agnostic rules, §4). A 40-case DS-1000 development subset drives the documentation track and is C1. The ledger controls author exposure; it cannot control whether the foundation models inside any engine saw these public repositories during training, a caveat shared by every arm.

Queries, corpus, metrics. Queries are the official ARB gold-free structured objects, serialized with sorted keys. File text comes from bare git clones at each case's `base_commit` (materialized snapshots; binary and non-UTF-8 blobs skipped). Metrics are the official ARB `sample_metrics` (MRR, Recall@k) plus BCY@8k, the budget-constrained yield from packing ranked files into an 8,000-token context. Documentation cases score *identifier hit*: whether the returned text contains the case's gold API identifier.

Freeze and attestation. All Delphi numbers come from one frozen commit (`91d76c1`) with exact vector scan, API top-k=20, and response-level serving-configuration attestation: every response carries the retrieval configuration actually served, and a run aborts on the first mismatch with its declared configuration. The frozen configuration is reproduced in Appendix A. Paired deltas carry cluster bootstrap 95% intervals (20,000 resamples, seed 1042; repository clusters for repository tracks, case clusters for documentation), plus exact McNemar tests on any-gold@20 movement.

Matched baseline ladder. The lexical comparators (whole-file BM25, ARB's official lexical ranker, and their development-tuned fusion) cannot say how much of a margin is Delphi's engineering and how much is a learned reranker on top of anything. We therefore add four conventional systems, each adding one Delphi component to the previous one, over the official ARB chunks (file chunk plus symbol chunks) of the identical corpus: **dense**, `text-embedding-3-small`—the embedding model inside Delphi—max-pooled to files; **hybrid**, reciprocal-rank fusion (k=60, equal weights, untuned) of the dense ranking and BM25; **+ rerankers**, the hybrid's top 50 narrowed to

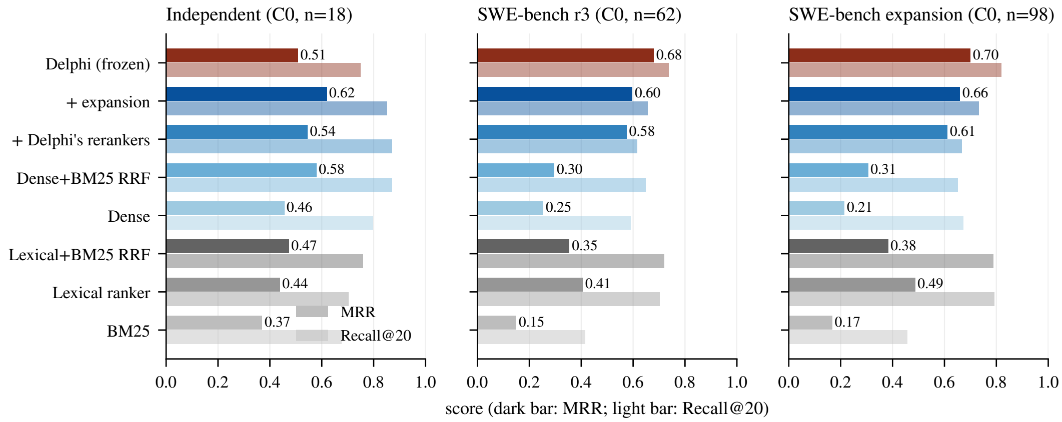

Figure 2: The three C0 sets, every system, scored once. Dark bars are MRR, light bars Recall@20. Grey: lexical comparators; blue: the conventional ladder, darker as components are added; red: the frozen Delphi stack. Full tables with Recall@5, BCY@8k, any-gold counts, latencies, and paired intervals are in Appendix D (Tables 5–7).

30 through Delphi's own cross-encoder (`ms-marco-MiniLM-L-6-v2`, blend 0.4) and then Delphi's own listwise `gpt-4o` reranker over the top 20 (seed 1042, identical prompt); **+ expansion**, the same with Delphi's hypothetical-document expansion (`gpt-4o-mini`, identical prompt and gate) feeding the dense query. The prompts and parsers are imported from Delphi's source so they are identical by construction (Appendix B). What remains between the last rung and Delphi is exactly Delphi's own contribution: exact-symbol, exact-path, path-affinity, and trigram branches; its chunker and index; tuned fusion weights; quoted-path demotion. Reported latencies exclude index construction for every system, as they did for Delphi.

Claim rule. "State of the art" would require leading every directly comparable engine on a C0 set with the complete comparable engine set runnable. No result in this paper meets it.

4 RQ2: REPOSITORY RETRIEVAL

Against lexical baselines. Delphi leads whole-file BM25, the official lexical ranker, and their tuned fusion on every set (Figure 2). On the independent final the Recall@20 leads over BM25 ($+0.074$ $[+0.019, +0.139]$) and the lexical ranker ($+0.046$ $[+0.009, +0.083]$) exclude zero while the fusion comparison is a tie; on SWE-bench MRR leads the strongest lexical system by $+0.274$ $[+0.194, +0.412]$ with Recall@20 tied; on the C2 ARB partition every lexical comparison excludes zero. None of this is in dispute, and none of it isolates Delphi's contribution.

Against the matched ladder. The independent final (Table 5) is the first of the two clean tests. The dense rung alone, using nothing but the embedding model Delphi already calls, ties Delphi on any-gold and exceeds it on Recall@20 (0.796 vs. 0.750). Adding BM25 by untuned reciprocal-rank fusion yields MRR 0.579 and Recall@20 0.870 against Delphi's 0.508 and 0.750; the Recall@20 gap (-0.120 $[-0.231, -0.019]$) excludes zero. Attaching Delphi's own rerankers and expansion reaches MRR 0.619 and BCY 0.546, and Delphi's MRR deficit against that rung (-0.111 $[-0.200, -0.034]$) also excludes zero. Six repository clusters is a small basis for an interval and we report the direction with that caveat; but the point is not that the conventional stack wins, it is that nothing Delphi adds on top of it is visible here.

The 62-instance SWE-bench set (Table 6) is the second clean test. Its point estimates lean the other way and its conclusion is the same. On issue text the dense rung alone is weak (MRR 0.254) and untuned fusion with BM25 helps little (0.296); Delphi's leads over both have intervals excluding zero. Attaching Delphi's rerankers to that hybrid moves MRR from 0.296 to 0.575, and expansion to 0.597: the reranking head, not the candidate branches, is where the margin over lexical systems comes from. Against the final rung Delphi leads by $+0.083$ MRR $[-0.107, +0.203]$,

Table 1: Pooled SWE-bench Verified localization, 160 C0 instances (Tables 6 and 7), 12 repositories. Paired Delphi−system deltas with repository-cluster bootstrap 95% intervals; * excludes zero.

Delphi − system	MRR	R@5	R@20	BCY@8k	any-gold (p)
Dense (same embedding)	+0.463* [+0.418, +0.547]	+0.365* [+0.296, +0.505]	+0.147* [+0.086, +0.277]	+0.490* [+0.434, +0.568]	131 vs 108 (p=0.00043)
Dense+BM25 RRF	+0.390* [+0.316, +0.492]	+0.242* [+0.154, +0.470]	+0.138* [+0.090, +0.226]	+0.440* [+0.394, +0.546]	131 vs 110 (p=0.0011)
+ Delphi's rerankers	+0.095* [+0.023, +0.181]	+0.116* [+0.064, +0.242]	+0.141* [+0.084, +0.248]	+0.108* [+0.064, +0.188]	131 vs 110 (p=0.0011)
+ expansion	+0.057 [−0.036, +0.114]	+0.069* [+0.015, +0.160]	+0.084* [+0.013, +0.154]	+0.076* [+0.017, +0.128]	131 vs 120 (p=0.08)
Lexical+BM25 RRF	+0.320* [+0.255, +0.434]	+0.216* [+0.151, +0.393]	+0.026 [−0.031, +0.128]	+0.334* [+0.286, +0.453]	131 vs 128 (p=0.69)
Lexical ranker	+0.237* [+0.174, +0.391]	+0.163* [+0.081, +0.368]	+0.030 [−0.019, +0.117]	+0.245* [+0.180, +0.406]	131 vs 128 (p=0.66)
BM25	+0.532* [+0.491, +0.630]	+0.494* [+0.411, +0.738]	+0.346* [+0.263, +0.538]	+0.521* [+0.478, +0.645]	131 vs 77 ($p=2.7e-12$)

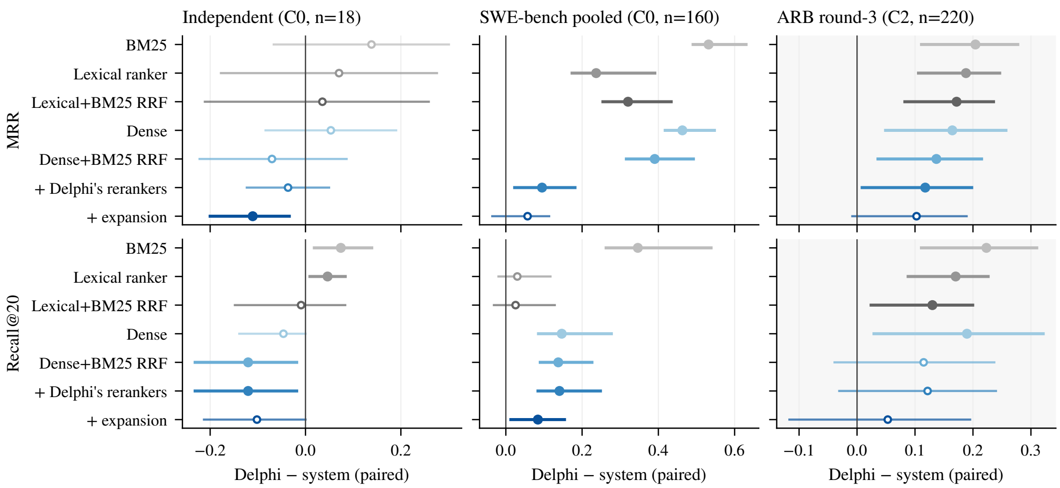

Figure 3: Paired Delphi−system differences with repository-cluster bootstrap 95% intervals (filled markers: interval excludes zero). Left: the 18-case independent final. Centre: the 160 pooled SWE-bench instances. Right, shaded: the C2 ARB partition, validation only. Recall@5 and BCY@8k panels are in Figure 7.

+0.080 Recall@20 [−0.105, +0.217], and +0.065 BCY [−0.062, +0.171], with any-gold 48 vs. 44 (p=0.42)—a tie. On the C2 ARB partition (Table 8) Delphi's margin over the final rung is +0.103 MRR [−0.008, +0.189] and +0.076 BCY [−0.019, +0.157], with Recall@5 and Recall@20 tied and any-gold 163 vs. 148 (p=0.08); against every earlier rung the leads exclude zero. That is the set on which Delphi's components were selected, so a margin there is the expected direction of selection, not evidence. Its workflow breakdown is still informative as a hypothesis (Appendix D, Table 10): Delphi's lead is concentrated in `trace2code` (MRR 0.579 vs. 0.149) and `edit2ripple`, whose queries carry file paths and symbol names that its exact-path, path-affinity, and symbol branches match directly, while on `code2test` and `comment2context` the conventional rung is equal or ahead. An exploratory split of the 62 SWE-bench issues by whether they contain a traceback or a Python path does not reproduce that pattern (Delphi's largest advantage is on issues with neither, n=35), so we record it as a hypothesis for a designed test, not a finding.

The expansion, and the pooled picture. To widen the C0 basis we drew 100 further Verified instances under a fresh salt from the 438 no round had used, applied two engine-agnostic rules before any scoring (a gold file must exist at the base commit, which removed one patch-created file from one instance's gold set; ARB's leakage check, which excluded two instances whose issue text carried patch markers), provisioned the frozen stack on the 98 remaining snapshots (index audit: 182,415 files, 1,079,743 chunks and embeddings, zero mismatches), and scored every system once (Table 7). The shape repeats: dense and untuned hybrid are weak on issue text, the reranking head does most of the work (0.306 → 0.611 MRR), expansion adds a little (0.660), and Delphi finishes at 0.700. On this larger set Delphi's margin over the final rung is +0.041 MRR [−0.028, +0.112], +0.061 Recall@5 [+0.023, +0.148], +0.087 Recall@20 [+0.042, +0.167], and +0.082 BCY [+0.030, +0.152]: three of four intervals exclude zero. Pooling the 160 SWE-bench instances (Table 1, Figure 3) gives the same reading, +0.057 MRR [−0.036, +0.114] with Recall@5, Recall@20, and BCY all excluding zero and any-gold 131 vs. 120 (p=0.08). Against the lexical ranker, which is strong at

Table 2: Like-for-like repeatability on ten documentation cases × ten runs (450 run pairs per engine). Retrieved-set columns use each engine's observable retrieval unit: retrieved items for Delphi and the documentation engine, the set of cited URLs for the synthesis engine. Byte-exact is the delivered context text; hit agreement is whether the identifier-hit outcome agreed across the pair.

Engine	Set exact	Order exact	Set Jaccard	Byte exact	Hit agreement
Delphi (raw context, k=5)	0.980	0.980	0.993	0.980	1.000
Documentation engine (quality-guided IDs)	0.542	0.362	0.835	0.356	0.918
Documentation engine (oracle IDs)	0.496	0.258	0.806	0.253	0.909
Synthesis engine (cited-URL set)	1.000	1.000	1.000	0.000	0.929

Recall@20 on issue text (0.793), Delphi's pooled Recall@20 lead is a tie ($+0.030$ [-0.019, $+0.117$]) while its MRR and BCY leads exclude zero. The rank distribution (Figure 8) shows where the difference sits: Delphi and the conventional stack place a gold file first on 62% and 58% of pooled instances, and Delphi misses the top 20 on 18% against 25%; the lexical ranker misses on 20% but ranks gold first on only 34%.

So the answer to the reviewers' question is not one number. On SWE-bench-style issue queries Delphi's own additions are worth roughly $+0.06$ to $+0.09$ on the yield metrics over a conventional stack built from its own parts, resolved at n=160; on the commit-to-files set they are worth less than nothing, resolved at n=18. In both cases the component that matters most is the one any practitioner would add first: a reranking head over a dense-plus-lexical candidate pool, which moves MRR by $+0.3$ where Delphi's remaining engineering moves it by $+0.06$. The price is latency: Delphi and the last two rungs answer in 2–6 seconds per query where dense retrieval answers in 0.5 (Appendix D, Table 11).

Development, briefly. The development partitions drove preregistered ablations whose outcomes we keep regardless of direction (Appendix E): exact vector scan replaced HNSW; expansion stayed on; a Gemini embedding swap and a file-level Okapi BM25 branch were rejected; two bugs surfaced by development queries (quoted-path retrieval and a generated-source exclusion that dropped a gold file) were fixed. All were selected on development or C2 evidence, which is exactly why they cannot be credited from the ARB partition.

5 RQ3: DETERMINISM, LIKE FOR LIKE

The pipeline was the variance. Repeating eight ARB queries ten times against the July stack gave a mean exact top-10 rate of 30.6%. Paired traces isolated uncached, unseeded LLM stages; insertion-order ties in fusion SQL; and approximate HNSW under concurrency. Total SQL ordering, single-flight sharing of concurrent identical hosted calls, an explicit seed, and a persistent stage cache take exact repeats to 100% in-process across two concurrent 8×10 packets and two 75-case repeats; a genuine cold restart still moved 7/8 lists until the cache was warm, after which 8/8 survive two restarts. We claim restart-durable repeatability from the first persisted answer, not that two clean installations agree on their first answer. Embedding APIs were not the problem: the small OpenAI model jitters at the fifth decimal and never changed top-ten membership across N=20 repeats; Gemini was bitwise deterministic (Appendix G).

Same quantity, every engine. Our earlier headline—98.0% byte-identical context for Delphi against 35.6% and 0.0% for the hosted engines—compared retrieval stability for two engines with generated-text stability for the third. Table 2 separates the layers. On retrieved-set stability the synthesis engine's cited sources are identical across every pair (1.000), Delphi's retrieved items across 98.0%, and the documentation engine's across 54% (quality-guided) or 50% (oracle IDs). What differs for the synthesis engine is the layer above retrieval: its answer text never repeats byte-for-byte, and that variation flips the identifier-hit outcome in 7.1% of pairs, against 0% for Delphi. The defensible statements are narrower than before: Delphi's caching and total ordering make its retrieved text repeatable, and repeatable retrieved text makes downstream decisions repeatable; the documentation engine's retrieval drifts; the synthesis engine's retrieval does not drift, its prose does. We have not

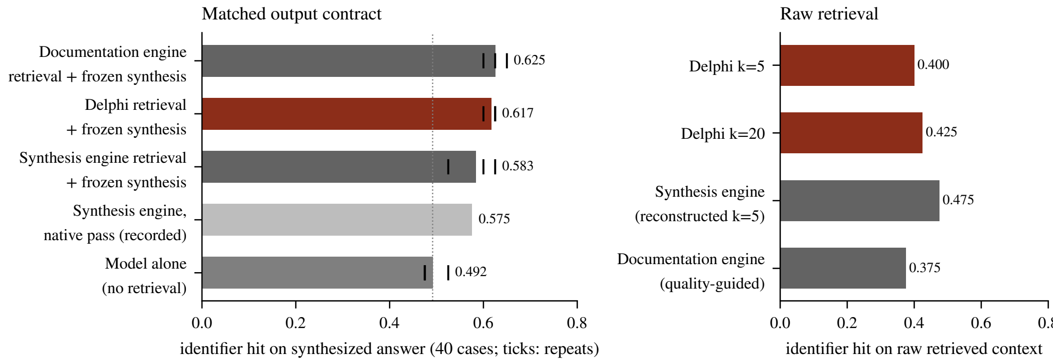

Figure 4: Documentation track, 40 development cases (C1). Left: every engine's retrieved evidence—including the synthesis engine's own five recorded sources—routed through one frozen synthesis stage (`gpt-5.4-mini`, one prompt, 8,000-token budget), three repeats (ticks); the dotted line is the no-retrieval floor. Right: the raw retrieved context scored directly. Tables with intervals are in Appendix F.

tested whether a caching wrapper would give the hosted engines the same repeatability, and expect that it would.

6 DOCUMENTATION: MATCHED CONTRACTS, ALL FOUR ARMS

Documentation retrieval is where hosted engines are strongest and where output contracts differ most. The synthesis engine returns a fluent answer with citations; the documentation engine and Delphi return raw context. Scored naively, the synthesis engine leads (identifier hit 0.575 against Delphi's 0.400 and the documentation engine's 0.375). That comparison conflates retrieval with the synthesis model's parametric knowledge.

Matching the contract, properly. Our first attempt routed the two raw-context engines through a frozen answer-style synthesis stage (`gpt-5.4-mini`, 1,600 output tokens, one system prompt) and compared them with the synthesis engine's *native* pass, which a reviewer correctly identified as only partially matched: the output format was aligned, the synthesis model and prompt were not. The engine's raw-retrieval mode returned no sources during the recorded round, so we could not route its retrieval at the time. Its synthesized responses, however, carry the five retrieved source chunks it grounded on. We reconstructed that context from the recorded responses (five sources per case, mean 2,754 tokens), packed it under the same 8,000-token budget with the same routine as every other arm, and routed it through the same frozen stage, three repeats, no new hosted calls. Figure 4 is now a four-arm comparison under one synthesis model, one prompt, one budget.

Three findings. First, on raw retrieved context the synthesis engine is ahead of Delphi: its reconstructed five-source context hits the gold identifier on 0.475 of cases against Delphi's 0.400 (k=5) and 0.425 (k=20). Second, under the matched contract the four arms finish within 0.05 of one another and every engine-versus-engine interval includes zero (Table 13); the synthesis engine's matched arm (0.583) is within 0.008 of its native pass, so its own synthesis added nothing our frozen stage did not. Third, the metric saturates: the bare model scores 0.492, so roughly 80% of every arm's score is parametric knowledge, and the retrieval-over-floor deltas are $+0.13$, $+0.13$, and $+0.09$. We do not detect a statistically resolved difference between any two engines under this evaluation. That is the supported sentence; we withdraw "parity or better," which an interval of $[-0.09, +0.16]$ does not establish. Establishing non-inferiority would require a predeclared margin and a sample several times this size.

7 RQ1: DOES ANY OF IT HELP AN AGENT?

Closed-tool seeding (a replication). ARB's own seed intervention holds a closed-tool agent fixed and varies its initial context. We replicated it with the policy model `gpt-5.4-mini`, tools

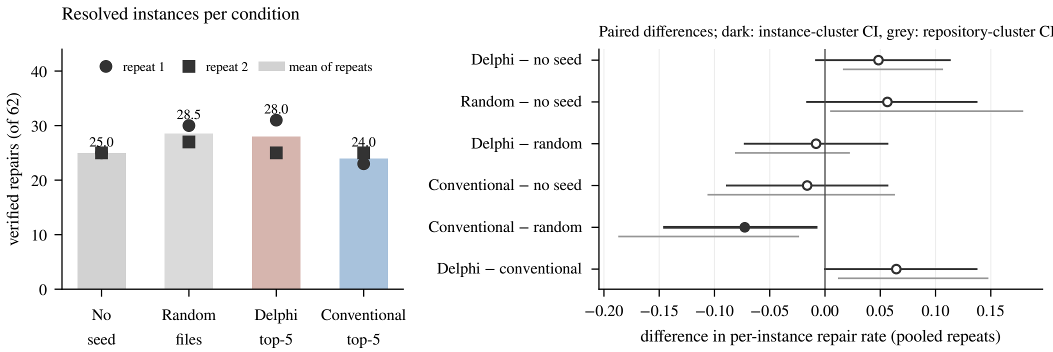

Figure 5: Executable pilot: 62 C0 SWE-bench Verified instances, mini-SWE-agent with `gpt-5.4-mini`, step budget 50, two independent trajectories per instance and condition. Left: verified repairs per condition and repeat. Right: paired differences in per-instance repair rate with instance-cluster (dark) and repository-cluster (grey) bootstrap intervals; the filled marker is the only interval excluding zero. Per-repeat tables and the protocol are in Appendix H.

`list_dir`/`grep`/`read_file`/`submit`, and 40 paired development cases per arm, adding a July-Delphi seed. File-F1: no seed 0.046; random files 0.013; BM25 seed 0.093; lexical seed 0.119; July-Delphi seed 0.234; oracle 0.858. The Delphi seed beats no-seed by $+0.188$ $[0.055, 0.294]$ and BM25 by $+0.142$ $[0.059, 0.208]$; against lexical the F1 interval includes zero $[-0.002, 0.190]$. Seeds change which files a frozen agent submits, ordered by seed quality, which is what the benchmark's authors also found. The oracle ceiling shows localization headroom in this harness; it does not show that retrieval dominates reasoning, editing, or validation failures in a repairing agent, and we withdraw that sentence from the earlier draft.

DS-1000 (unresolved, not negative). With the frozen generator, 40 cases $\times$ 3 repeats: no retrieval 0.575; documentation engine 0.567 (-0.008 $[-0.142, 0.125]$); Delphi retrieval+synthesis 0.592 ($+0.017$ $[-0.125, 0.158]$); synthesis engine 0.642 ($+0.067$ $[-0.075, 0.200]$). Every interval spans effects from substantial harm to substantial benefit. This experiment does not show that retrieval fails to help; it shows that 40 cases cannot tell.

Executable pilot (preregistered). The reviewers' request was direct: measure retrieval and executable outcome on the same tasks. We preregistered a pilot (Appendix H) on the 62 C0 SWE-bench instances whose rankings every system in Table 6 had already produced once. The agent is mini-SWE-agent with `gpt-5.4-mini`, ordinary shell tools in every condition, and a fixed step budget; a seed is a block appended to the issue naming the top five files a recorded ranking produced, with the head of each file, capped at 3,000 tokens. Conditions: no seed; five random non-gold files in the same block (prompting control); Delphi's recorded top five; the conventional ladder's recorded top five. The outcome is verified repair under the official harness, paired by instance, with instance- and repository-cluster intervals. Every condition was run twice (two independent trajectories per instance); Figure 5 reports both repeats and their pooled per-instance means.

The two repeats disagree by more than the effects under study, which is the first result. Resolved counts were 25/30/31/23 (none/random/Delphi/conventional) in the first repeat and 25/27/25/25 in the second: the Delphi seed's six-instance lead over no seed in the first repeat vanished in the second, and the conventional seed's eight-instance deficit shrank to zero. Pooled, no seed effect is resolved. A Delphi seed changes the per-instance repair rate by $+0.048$ $[-0.008, +0.113]$ against no seed, and by -0.008 $[-0.073, +0.056]$ against a block of five random non-gold files, which itself sits at $+0.056$ $[-0.016, +0.137]$ over no seed. Whatever the block contributes at this size, it is not separable from the value of naming the right files. The conventional ladder's seed, whose top five named a gold file on 42 instances against Delphi's 47, is the only arm that trails a comparator with an interval excluding zero (-0.073 $[-0.145, -0.008]$ against the random control); we do not have an explanation and offer none. Budgets were not binding: agents used 7–8 of 50 steps and cost about two cents per instance; the seed's tokens raised cost by roughly 10–30%, and Delphi's seed reduced steps slightly relative to the random block. The pilot's width is its finding: at $n{=}62$ with two trajectories, effects of ± 0.05 in verified repair are inside sampling noise, so the study that could answer the title question needs several hundred instances and more repeats, which the per-run cost (about $11 for all 496 trajectories) makes affordable.

8 The hosted repository arm

The synthesis engine also markets a repository surface. Scoring it requires 68 exact `(repo, base_commit)` snapshots in the provider's account for the development split alone. Whole-snapshot ingestion of large repositories stalled in a non-terminal `syncing` state; 32/68 pairs remain indexed from an earlier sharded round; one-shard probes in August and again on 2026-09-09 under a fresh credential never left `syncing` within their 40-minute deadlines (Appendix I). No repository score is reported for that engine in either direction. This is a gap in the evidence, not a result, and after the matched-ladder findings it is no longer the most important gap.

9 What we claim, and what we do not

Claimed. (1) Delphi leads whole-file BM25 and the official lexical ranker on every set, with intervals excluding zero on the C0 sets for Recall@20 (independent) and for MRR and BCY (SWE-bench, 160 pooled); its Recall@20 lead over the lexical ranker on SWE-bench is a tie. (2) Against a conventional dense+BM25 hybrid built from Delphi's own embedding model with Delphi's own rerankers and expansion attached, Delphi leads by $+0.06$–0.08 on Recall@5, Recall@20, and BCY over 160 pooled SWE-bench instances (intervals excluding zero; MRR tied), and trails on the 18-case independent final (two intervals excluding zero). The reranking head accounts for most of the margin over lexical systems on every set. (3) Under one frozen synthesis stage, no two documentation engines are statistically distinguishable at $n{=}40$, and all sit 0.08–0.13 above a no-retrieval floor. (4) Delphi's retrieved text is repeatable in-process and across restarts with a warm cache; on retrieved-set stability the hosted synthesis engine is at least as stable. (5) Retrieval seeds change a closed-tool agent's file selection, ordered by seed quality, replicating ARB.

Not claimed. (1) State of the art, anywhere. (2) Any confirmatory result on the ARB round-3 partition, which is C2. (3) Documentation parity or non-inferiority. (4) Downstream utility on DS-1000 at $n{=}40$, or in executable repair at $n{=}62$, in either direction. (5) That Delphi's structural branches, tuned weights, or quoted-path demotion add value in general: they separate from the conventional ladder on issue-style queries and not on commit-to-files queries, and the mechanism is untested.

Limitations. One engine is studied in depth; the hosted engines are studied only where their surfaces permitted a matched measurement, and one arm is missing. The C0 sets are small (18, 62, 98) and drawn from two query styles; the independent final has six repository clusters. Every arm shares one embedding model and one family of foundation models, so training-set exposure of the public repositories is uncontrolled. The executable pilot uses a small policy model and a single agent scaffold, and its repeats show that two trajectories per cell are too few. The documentation metric is a surface identifier match, which saturates. The exposure ledger records what the authors could see; it does not audit what the models had seen.

10 Conclusion

Measured against baselines built from its own parts, and on cases its development could not see, most of Delphi's advantage is the part any careful practitioner would assemble: an embedding model, BM25, reciprocal-rank fusion, a small cross-encoder, a listwise reranker, and query expansion. What is Delphi's own is worth a further $+0.06$ to $+0.08$ on yield metrics for issue-style queries at $n{=}160$, and nothing on commit-to-files queries at $n{=}18$. The claims we made from the re-partitioned ARB set did not survive the ledger that classified how that set had been used, and the claims we made against hosted engines did not survive matching the quantity being measured. What survives is a protocol—exposure classes at the case level, component-matched baselines, matched output contracts, like-for-like determinism, artifact-generated tables and figures—and the open question the pilot begins to answer: whether any of this changes what an agent repairs.

AI USE STATEMENT

Generative AI tools were used to help formulate experimental questions, write and test evaluation and product code, operate experiments, inspect failures, search for related work, and draft and edit this manuscript. They were not used to generate benchmark gold labels. Empirical claims are checked against persisted run artifacts and canonical repository snapshots; every table and figure in this paper is generated from those artifacts by scripts in the released archive. All AI-assisted work remains subject to human review before submission, and the authors take responsibility for the final content, claims, code, and artifacts.

ETHICS STATEMENT

The study uses public open-source repositories and public benchmark records. Repository contents are processed locally except where a named hosted retrieval or model API is itself the evaluated system. We retain commit identifiers and public issue or review text for provenance, but do not intentionally collect private repository data or credentials. Because benchmark results can be used in product marketing, we predeclare the claim rule, preserve failed runs, publish the exposure ledger, and give losses the same prominence as wins.

REPRODUCIBILITY STATEMENT

The archive accompanying this paper (Appendix K) contains the split manifests and case-level exposure ledger (Appendix C), every per-case artifact (queries, gold paths, ranked outputs, synthesized answers, latencies), every paired-analysis JSON, the harness code for every system and comparator including the matched ladder (Appendix B), the frozen serving configuration and the scripts that launch it (Appendix A), the pilot protocol, datasets, and all 496 agent trajectories with harness verdicts (Appendix H), the prompts used by every LLM stage verbatim (Appendices B, F, H), and the scripts that regenerate every table and figure in this paper from those artifacts. Repository corpora are re-cloned from public GitHub at the recorded commits by a provided script; hosted-engine runs require the reader's own credentials.

REFERENCES

Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided llm agents for code localization. In *ACL*, pp. 8697–8727, 2025. doi: 10.18653/v1/2025.acl-long.426.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *ACL*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *SIGIR*, 2020.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*, 2026.

Bowen Qin and Yi Xie. Agent retrieval bench: Evaluating repository context retrieval for coding agents. *arXiv preprint arXiv:2607.24882*, 2026.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In *EMNLP*, pp. 2471–2484, 2023. doi: 10.18653/v1/2023.emnlp-main.151.

Fuwei Zhang, Yanzhao Zhang, Mingxin Li, Dingkun Long, Lexiang Hu, Pengjun Xie, Zhao Zhang, and Fuzhen Zhuang. Core-bench: A comprehensive benchmark for code retrieval in the era of agentic coding. *arXiv preprint arXiv:2606.11864*, 2026a.

Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-explore: Benchmarking how coding agents explore repositories. *arXiv preprint arXiv:2606.07297*, 2026b.

Sheng Zhang, Yifan Ding, Shuquan Lian, Shun Song, and Hui Li. Coderag: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. In *EMNLP*, pp. 23278–23288, 2025. doi: 10.18653/v1/2025.emnlp-main.1187.

APPENDIX

The appendix documents every experiment in enough detail to rerun it from the released archive. Appendix A gives the frozen Delphi configuration; B the matched baseline ladder and the verbatim prompts it shares with Delphi; C the exposure ledger and the round chronology; D the complete repository-retrieval tables and additional figures; E the development-time ablations and the rejected ideas; F the documentation track; G the determinism probes; H the executable pilot protocol, seed block, and per-repeat results; I the hosted engine accounting; J compute and cost; K the artifact release; and L a crosswalk from our earlier report's claims to their corrected form.

A FROZEN SYSTEM SPECIFICATION

Frozen commit `91d76c1` of the Delphi repository (branch `feature/generated-source-freeze`), served natively against PostgreSQL 14 with pgvector from a git worktree pinned to that commit. Every scored response carried the following attested configuration: embedding model `text-embedding-3-small`; vector mode exact scan (HNSW disabled for scoring; `ef_search` 100 when enabled); hybrid candidate branches vector, BM25, exact symbol, exact path, path affinity, and trigram with reciprocal-rank fusion weights 0.50/0.25/0.20/0.15/0.15/0.05 and k=60; 50 fused candidates; cross-encoder `cross-encoder/ms-marco-MiniLM-L-6-v2` over the top 30 with blend α=0.4 (code-aware reranker disabled); quoted-path demotion when the query envelope carries evidence keys; listwise reranking of the top 20 by `gpt-4o` at temperature 0, seed 1042, 280-character excerpts, 300-token replies; hypothetical-document expansion by `gpt-4o-mini` (temperature 0, seed 1042, 220-token replies) gated on prose-like queries and feeding only the vector branch; persistent SQLite stage cache; API top-k=20; file-diverse BM25 and path-token branches disabled; agent quality mode with tests, docs, examples, configs, and manifests indexed and generated source retained under 500,000-byte and NUL-content guards. Context packing for BCY@8k uses the official ARB `pack_files` at an 8,000-token budget.

Table 3: Environment of the frozen serving stack (`stack/native-r4-frozen.sh`). Secrets and local paths omitted.

Variable	Value
EMBEDDING_PROVIDER	openai
OPENAI_EMBEDDING_MODEL	text-embedding-3-small
SYNSC_VECTOR_EXACT_SCAN	true (HNSW not consulted)
SYNSC_HNSW_EF_SEARCH	100 (inert under exact scan)
SYNSC_HYBRID_CANDIDATES	50
SYNSC_HYBRID_RERANK_K	30
SYNSC_ENABLE_RERANKER	true (cross-encoder/ms-marco-MiniLM-L-6-v2)
SYNSC_USE_CODE_RERANKER	false
RERANKER_BLEND_ALPHA	0.4
SYNSC_QUERY_EXPANSION	true (gpt-4o-mini, gated)
SYNSC_LISTWISE_RERANK	true
SYNSC_LISTWISE_RERANK_MODEL	gpt-4o
SYNSC_LISTWISE_RERANK_K	20
SYNSC_LLM_SEED	1042
SYNSC_LLM_CACHE_ENTRIES	2048 (in-memory tier)
SYNSC_LLM_CACHE_DB	persistent SQLite stage cache (archived)
SYNSC_FILE_DIVERSE_BM25	false
SYNSC_WORKER_THREADS	4
SYNSC_RATE_LIMIT_SEARCH, SYNSC_RATE_LIMIT_INDEX	10000/minute (local only)

Attestation. Every `/search` response from the frozen stack carries a `retrieval_config` object listing the branches, weights, reranker, listwise, expansion, vector mode, and k actually used to serve it. The runner compares this object with the run's declared configuration on every response and aborts the run on the first mismatch; each summary records `retrieval_configuration_verified` and the set of observed configurations, which is a singleton in every reported run.

Index audit. Before the expansion set was scored, an audit walked every provisioned snapshot and compared the file count at the base commit (after the same size and binary guards) with the indexed file, chunk, and embedding counts in the database: 98 snapshots, 182,415 files, 1,079,743 chunks, 1,079,743 embeddings, zero mismatches. A gold-searchability preflight then confirmed that every gold path of every case is present in the index; it is this preflight that surfaced the one patch-created gold file (§4).

B MATCHED BASELINE LADDER

Corpus and chunks. Every rung operates on the same materialized snapshot as Delphi and the lexical comparators, chunked with the official ARB routine (one whole-file chunk plus one chunk per top-level symbol). Non-UTF-8 and binary blobs are skipped by the same guard Delphi uses.

Dense. Chunks are embedded with `text-embedding-3-small` (1,536 dimensions) in token-aware batches (at most 200,000 tokens or 512 texts per request; texts above 8,000 tokens are truncated with `cl100k_base`); vectors are cached in SQLite keyed by model and content hash, so a chunk is embedded once regardless of how many rungs read it. Queries are embedded with the same model; similarity is cosine; a file's score is the maximum over its chunks; the top 50 files are returned.

Hybrid. ARB's official BM25 chunk ranker over the same chunks, max-pooled to files, fused with the dense file ranking by reciprocal-rank fusion, $\mathrm{RRF}(f) = \sum_r 1/(60 + \mathrm{rank}_r(f))$ with equal weights. Nothing is tuned.

+ Delphi's rerankers. The top 50 fused files are narrowed to 30 with Delphi's cross-encoder (`ms-marco-MiniLM-L-6-v2`, scored on the query and the first 4,000 characters of the file's best-matching chunk, its sigmoid blended at $\alpha{=}0.4$ with the fused score normalized by the top fused score, as in Delphi), then the top 20 are reordered by Delphi's listwise reranker (`gpt-4o`, temperature 0, seed 1042, 280-character excerpts). The system prompt and the reply parser are imported from Delphi's `synsc/services/listwise_rerank.py` at the frozen commit.

+ Delphi's expansion. Delphi's hypothetical-document expansion (`gpt-4o-mini`, temperature 0, seed 1042, 220-token reply) is applied under Delphi's own gate (prose-like queries only) and the expansion text is appended to the dense query; BM25 sees the original query, as in Delphi. The prompt is imported from `synsc/services/query_expansion.py`.

Caching and latency. LLM replies are cached in SQLite keyed by model, prompt, and seed, so a repeat of a rung is bitwise identical and free. Reported latencies are wall-clock per query on a warm process excluding index construction, as for Delphi, and include hosted API round trips.

Listwise reranker system prompt (verbatim, shared with Delphi).

```
You rank candidate source files by how directly each one
answers a developer's request about a codebase.  You are
given a request and a numbered list of candidates, each with
its repository path and an excerpt.  Return the candidate
numbers ordered best first, as a JSON array of integers, and
nothing else.  Example:  [4,1,9,2] Include every candidate
number exactly once.  Judge by what the request actually
asks for.  If it asks which test covers some behaviour, a
test file is the answer and the implementation is not.  If
it asks where something is implemented, the reverse holds.
Prefer the file that would have to be opened or edited to
satisfy the request over one that merely mentions the same
words.
```

Query-expansion system prompt (verbatim, shared with Delphi).

```
You write short, plausible code snippets that would answer a
developer's question about an unfamiliar codebase.  Reply
with code only:  no prose, no explanation, no fences.
```

```
Invent idiomatic function, class, and variable names for
the language implied by the question.  Being wrong about the
specifics is fine; using the vocabulary real code would use
is what matters.  Keep it under 15 lines.
```

C Exposure ledger and chronology

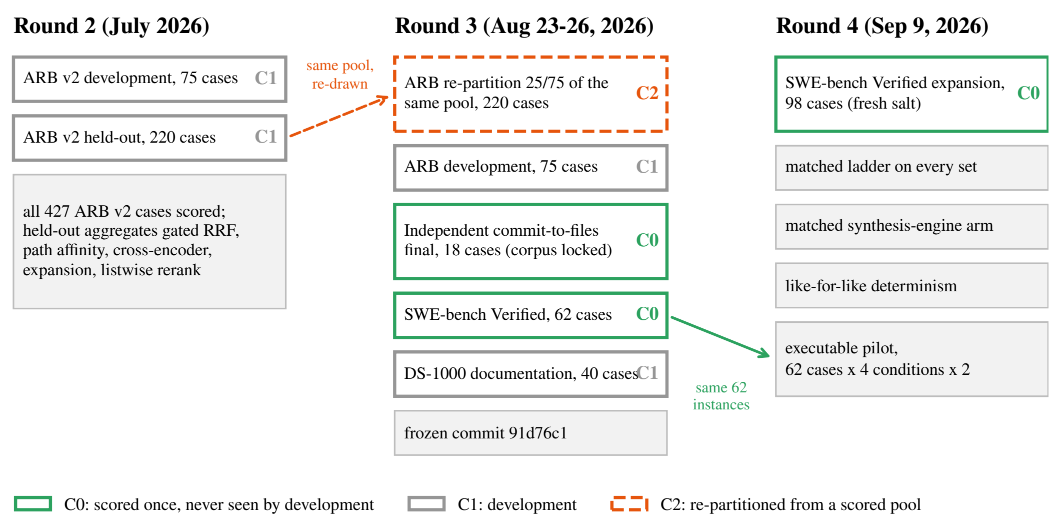

Figure 6: Evaluation rounds, the sets each round scored, and their exposure class. Round 2 scored every case of the ARB v2 pool and used the held-out aggregates to accept components; round 3 re-drew a 220-case partition from that pool (C2) and introduced the C0 sets; round 4 added the matched ladder, the 98-case expansion, the matched synthesis-engine arm, like-for-like determinism, and the executable pilot on the round-3 62-instance set.

Table 4: Case-level exposure ledger (`results/exposure-ledger-v1.json`). Every case ID of every set is listed in the machine-readable file with the evidence for its class.

Set	Cases	Class	Supports
ARB round-3 partition (positive workflows)	220	C2	development/validation evidence; not a confirmatory test of the frozen stack
ARB round-3 development	75	C1	development
Independent commit-to-files final	18	C0	confirmatory
Independent commit-to-files development	48	C1	development
SWE-bench Verified, round 3	62	C0	confirmatory
SWE-bench Verified, round-4 expansion	98	C0	confirmatory
DS-1000 documentation subset	40	C1	development

Decisions and the evidence that informed them. Query-time embedding alignment, RRF fusion, the path-affinity branch, the cross-encoder blend, expansion, and listwise reranking were accepted on round-2 development (75) and round-2 held-out (220) aggregates over the ARB v2 pool that round 3 re-partitioned (Appendix E); quoted-path demotion on July per-workflow rates over all 427 cases; exact scan, single-flight, the persistent cache, and total SQL ordering on round-3 development and determinism probes; generated-source indexing on the gold-path preflight over the independent and ARB development sets; the File-Okapi rejection on both development sets. No decision was informed by any C0 set: the independent final corpus was locked before any query was issued; the 62 SWE-bench instances and the 98 expansion instances were each scored once per system after the frozen commit was fixed.

Why re-drawing does not un-expose. A case scored in round 2 contributed to the aggregate that accepted or rejected a component. Any partition drawn later from the same pool inherits that influence at the aggregate level, whatever the seed. What a fresh draw does exclude is per-case tuning, and only if per-case outputs were never inspected; the round-2 per-case outputs are not in the archive, so we cannot demonstrate that from the record, and we do not claim it.

What the ledger cannot control. The foundation models inside every arm (OpenAI embeddings, `gpt-4o`, `gpt-4o-mini`, `gpt-5.4-mini`, and the models behind the hosted engines) were trained on public code that very likely includes these repositories and possibly the fixing commits. This affects every arm and every rung equally within an experiment, but it means absolute numbers are not estimates of performance on unseen code.

D COMPLETE REPOSITORY-RETRIEVAL RESULTS

Table 5: Independent commit-to-files final (C0): 18 cases, 6 repositories, scored once. Any-gold is the number of cases with a gold file in the top 20; latency is the median query time in seconds. Δ rows are Delphi minus the named system with repository-cluster bootstrap 95% intervals; * excludes zero.

System	MRR	R@5	R@20	BCY@8k	any-gold	s
Delphi (frozen)	**0.508**	**0.551**	**0.750**	**0.454**	15/18	3.7
Dense (same embedding)	0.455	0.523	0.796	0.347	15/18	0.5
Dense+BM25 RRF	0.579	0.639	0.870	0.403	16/18	0.2
+ Delphi's rerankers	0.545	0.644	0.870	0.454	16/18	2.3
+ expansion	0.619	0.662	0.852	0.546	16/18	4.1
Lexical+BM25 RRF	0.473	0.500	0.759	0.287	15/18	1.6
Lexical ranker	0.438	0.403	0.704	0.259	14/18	1.1
BM25	0.370	0.458	0.676	0.259	14/18	1.2
Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero						
Δ vs. Dense (same embedding)	+0.053 [−0.084, +0.190]	+0.028 [−0.148, +0.194]	−0.046 [−0.139, +0.000]	+0.106 [−0.074, +0.259]	15 vs 15	p=1
Δ vs. Dense+BM25 RRF	−0.070 [−0.222, +0.086]	−0.088 [−0.236, +0.056]	−0.120* [−0.231, −0.019]	+0.051 [−0.120, +0.222]	15 vs 16	p=1
Δ vs. + Delphi's rerankers	−0.037 [−0.123, +0.049]	−0.093 [−0.324, +0.083]	−0.120* [−0.231, −0.019]	+0.000 [−0.139, +0.194]	15 vs 16	p=1
Δ vs. + expansion	−0.111* [−0.200, −0.034]	−0.111 [−0.324, +0.046]	−0.102 [−0.213, +0.000]	−0.093 [−0.278, +0.074]	15 vs 16	p=1
Δ vs. Lexical+BM25 RRF	+0.035 [−0.211, +0.259]	+0.051 [−0.088, +0.194]	−0.009 [−0.148, +0.083]	+0.167 [−0.074, +0.472]	15 vs 15	p=1
Δ vs. Lexical ranker	+0.070 [−0.177, +0.276]	+0.148 [−0.046, +0.343]	+0.046* [+0.009, +0.083]	+0.194 [−0.046, +0.481]	15 vs 14	p=1
Δ vs. BM25	+0.138 [−0.067, +0.301]	+0.093 [−0.037, +0.278]	+0.074* [+0.019, +0.139]	+0.194 [−0.046, +0.481]	15 vs 14	p=1

Table 6: SWE-bench Verified localization, round 3 (C0): 62 instances, 12 repositories, scored once. Columns as in Table 5.

System	MRR	R@5	R@20	BCY@8k	any-gold	s
Delphi (frozen)	**0.680**	**0.704**	**0.737**	**0.638**	48/62	3.6
Dense (same embedding)	0.254	0.324	0.591	0.210	39/62	0.5
Dense+BM25 RRF	0.296	0.452	0.648	0.234	43/62	2.0
+ Delphi's rerankers	0.575	0.598	0.616	0.533	41/62	3.7
+ expansion	0.597	0.623	0.657	0.573	44/62	6.2
Lexical+BM25 RRF	0.354	0.500	0.719	0.306	47/62	13.2
Lexical ranker	0.406	0.509	0.702	0.387	46/62	11.0
BM25	0.149	0.230	0.416	0.145	29/62	10.7
Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero						
Δ vs. Dense (same embedding)	+0.426* [+0.328, +0.492]	+0.380* [+0.262, +0.583]	+0.146 [+0.000, +0.288]	+0.427* [+0.340, +0.500]	48 vs 39	p=0.064
Δ vs. Dense+BM25 RRF	+0.384* [+0.275, +0.464]	+0.252* [+0.152, +0.442]	+0.089 [−0.038, +0.182]	+0.404* [+0.288, +0.517]	48 vs 43	p=0.3
Δ vs. + Delphi's rerankers	+0.104 [−0.065, +0.210]	+0.106 [+0.000, +0.263]	+0.121 [−0.033, +0.259]	+0.105 [+0.000, +0.190]	48 vs 41	p=0.14
Δ vs. + expansion	+0.083 [−0.107, +0.203]	+0.081 [−0.056, +0.251]	+0.080 [−0.105, +0.217]	+0.065 [−0.062, +0.171]	48 vs 44	p=0.42
Δ vs. Lexical+BM25 RRF	+0.325* [+0.216, +0.453]	+0.204* [+0.099, +0.455]	+0.018 [−0.073, +0.075]	+0.331* [+0.211, +0.552]	48 vs 47	p=1
Δ vs. Lexical ranker	+0.274* [+0.194, +0.412]	+0.195* [+0.107, +0.403]	+0.035 [−0.042, +0.111]	+0.251* [+0.157, +0.433]	48 vs 46	p=0.75
Δ vs. BM25	+0.530* [+0.450, +0.662]	+0.474* [+0.352, +0.726]	+0.321* [+0.189, +0.572]	+0.493* [+0.395, +0.669]	48 vs 29	p=0.00055

Exploratory split of the SWE-bench issues. To test the workflow hypothesis outside the C2 set we split the 62 round-3 issues by whether the issue text contains a Python traceback or a path ending in `.py`. Delphi's MRR lead over the final conventional rung was largest on issues with neither (n=35), and smaller on issues with a traceback or a path (n=27). The split was decided after the rankings existed and is reported as exploratory; it does not support the hypothesis that Delphi's structural branches are what separate it on issue-style queries.

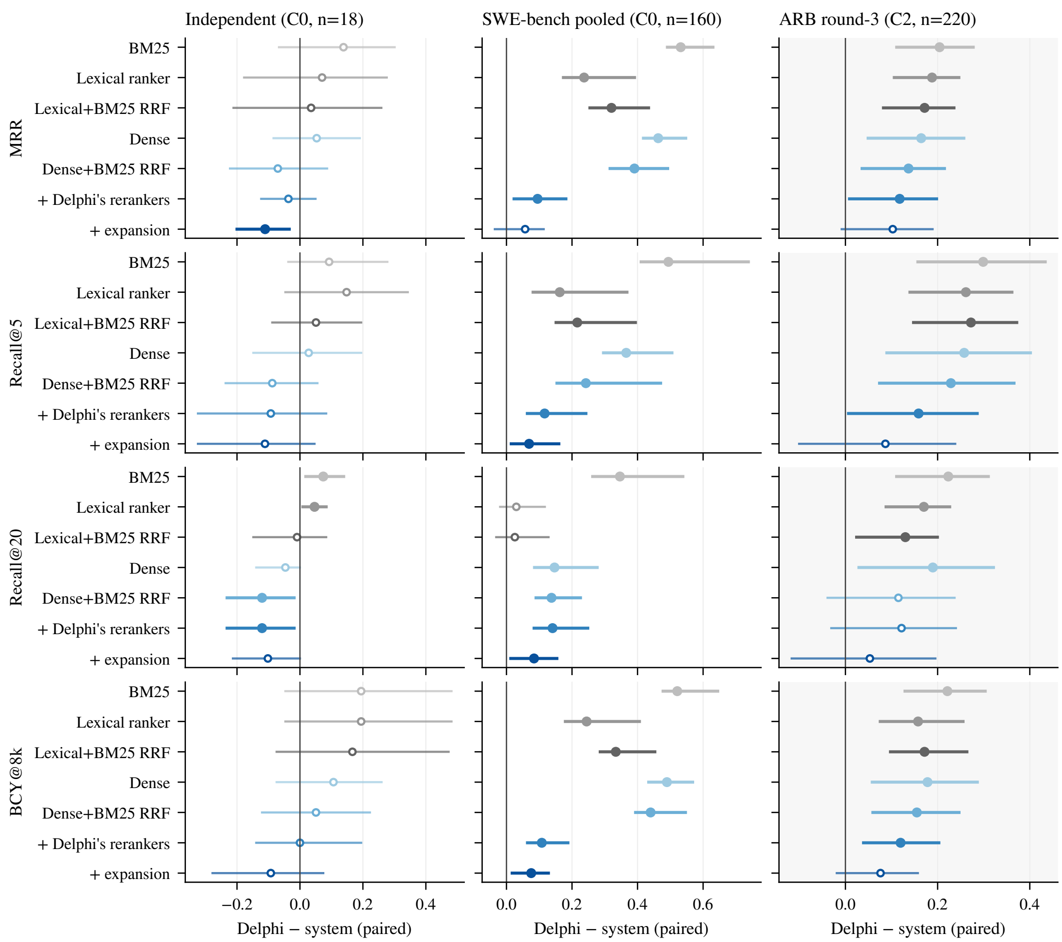

Figure 7: Paired Delphi−system differences on all four metrics with repository-cluster bootstrap 95% intervals (20,000 resamples, seed 1042). Filled markers: interval excludes zero. Left: independent final (C0, n=18, 6 clusters). Centre: pooled SWE-bench (C0, n=160, 12 clusters). Right, shaded: ARB round-3 partition (C2, n=220, 19 clusters), validation only.

Table 7: SWE-bench Verified expansion (C0): 98 further instances drawn under a fresh salt and never used in any round, 11 repositories, scored once from the frozen commit after the index audit and the gold-searchability preflight. Columns as in Table 5.

System	MRR	R@5	R@20	BCY@8k	any-gold	s
Delphi (frozen)	**0.700**	**0.745**	**0.820**	**0.685**	83/98	6.4
Dense (same embedding)	0.214	0.389	0.673	0.156	69/98	0.8
Dense+BM25 RRF	0.306	0.509	0.651	0.223	67/98	2.0
+ Delphi's rerankers	0.611	0.622	0.667	0.576	69/98	3.8
+ expansion	0.660	0.684	0.733	0.603	76/98	6.2
Lexical+BM25 RRF	0.383	0.521	0.789	0.350	81/98	4.2
Lexical ranker	0.487	0.603	0.793	0.445	82/98	2.2
BM25	0.168	0.238	0.457	0.146	48/98	2.2
Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero						
Δ vs. Dense (same embedding)	+0.486* [+0.434, +0.622]	+0.356* [+0.281, +0.496]	+0.147* [+0.083, +0.330]	+0.529* [+0.453, +0.656]	83 vs 69	p=0.0043
Δ vs. Dense+BM25 RRF	+0.395* [+0.299, +0.552]	+0.236* [+0.133, +0.513]	+0.168* [+0.116, +0.307]	+0.463* [+0.391, +0.636]	83 vs 67	p=0.0015
Δ vs. + Delphi's rerankers	+0.089* [+0.022, +0.227]	+0.123* [+0.067, +0.277]	+0.153* [+0.099, +0.298]	+0.110* [+0.062, +0.238]	83 vs 69	p=0.0043
Δ vs. + expansion	+0.041 [−0.028, +0.112]	+0.061* [+0.023, +0.148]	+0.087* [+0.042, +0.167]	+0.082* [+0.030, +0.152]	83 vs 76	p=0.17
Δ vs. Lexical+BM25 RRF	+0.317* [+0.229, +0.466]	+0.224* [+0.141, +0.415]	+0.031 [−0.045, +0.205]	+0.335* [+0.247, +0.472]	83 vs 81	p=0.79
Δ vs. Lexical ranker	+0.213* [+0.135, +0.404]	+0.142* [+0.050, +0.396]	+0.027 [−0.035, +0.163]	+0.241* [+0.150, +0.439]	83 vs 82	p=1
Δ vs. BM25	+0.533* [+0.453, +0.668]	+0.507* [+0.405, +0.750]	+0.362* [+0.279, +0.554]	+0.539* [+0.455, +0.712]	83 vs 48	p=5.5$e-10$

Table 8: ARB round-3 partition (C2, validation evidence only): 220 cases, 19 repositories. Columns as in Table 5. Reported for completeness; no confirmatory claim rests on it.

System	MRR	R@5	R@20	BCY@8k	any-gold	s
Delphi (frozen)	**0.332**	**0.476**	**0.648**	**0.296**	163/220	4.3
Dense (same embedding)	0.168	0.218	0.458	0.118	115/220	0.5
Dense+BM25 RRF	0.195	0.247	0.533	0.141	137/220	0.5
+ Delphi's rerankers	0.214	0.317	0.526	0.176	135/220	2.1
+ expansion	0.229	0.389	0.595	0.220	148/220	4.2
Lexical+BM25 RRF	0.160	0.204	0.518	0.124	131/220	1.9
Lexical ranker	0.144	0.215	0.478	0.138	123/220	1.4
BM25	0.128	0.177	0.425	0.075	113/220	1.3
Paired Delphi−system deltas, repository-cluster bootstrap 95% intervals; * interval excludes zero						
Δ vs. Dense (same embedding)	+0.164* [+0.049, +0.257]	+0.258* [+0.090, +0.401]	+0.190* [+0.029, +0.321]	+0.178* [+0.058, +0.286]	163 vs 115	p=5.9$e-08$
Δ vs. Dense+BM25 RRF	+0.137* [+0.036, +0.215]	+0.229* [+0.074, +0.366]	+0.115 [−0.039, +0.237]	+0.155* [+0.060, +0.246]	163 vs 137	p=0.0022
Δ vs. + Delphi's rerankers	+0.118* [+0.009, +0.198]	+0.159* [+0.007, +0.286]	+0.122 [−0.031, +0.240]	+0.120* [+0.039, +0.203]	163 vs 135	p=0.0011
Δ vs. + expansion	+0.103 [−0.008, +0.189]	+0.087 [−0.100, +0.238]	+0.053 [−0.117, +0.195]	+0.076 [−0.019, +0.157]	163 vs 148	p=0.082
Δ vs. Lexical+BM25 RRF	+0.172* [+0.083, +0.235]	+0.272* [+0.148, +0.372]	+0.130* [+0.024, +0.200]	+0.172* [+0.098, +0.263]	163 vs 131	p=0.00017
Δ vs. Lexical ranker	+0.188* [+0.106, +0.246]	+0.261* [+0.140, +0.361]	+0.170* [+0.088, +0.226]	+0.158* [+0.076, +0.255]	163 vs 123	p=2.4$e-06$
Δ vs. BM25	+0.204* [+0.111, +0.277]	+0.299* [+0.157, +0.433]	+0.223* [+0.111, +0.310]	+0.221* [+0.129, +0.303]	163 vs 113	p=5$e-09$

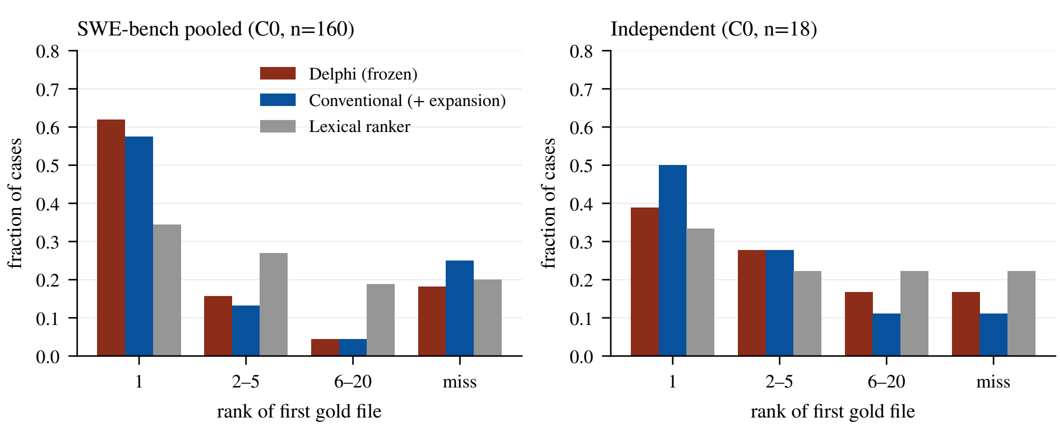

Figure 8: Rank of the first gold file for Delphi, the final conventional rung, and the lexical ranker. Left: 160 pooled SWE-bench instances. Right: the 18-case independent final. "Miss" is no gold file in the top 20.

Table 9: Pooled SWE-bench Verified (160 C0 instances) by repository. Means of per-instance metrics; repositories ordered by instance count. Repository-cluster intervals in Table 1 resample these twelve clusters.

Repository	n	Delphi		Conventional (+ expansion)		Lexical ranker	
		MRR	R@20	MRR	R@20	MRR	R@20
`django/django`	73	0.691	0.792	0.618	0.696	0.518	0.779
`sympy/sympy`	24	0.729	0.779	0.663	0.739	0.608	0.773
`sphinx-doc/sphinx`	14	0.615	0.643	0.607	0.607	0.227	0.679
`matplotlib/matplotlib`	11	0.682	0.818	0.530	0.727	0.223	0.500
`scikit-learn/scikit-learn`	9	0.815	0.889	0.623	0.722	0.396	0.889
`astropy/astropy`	7	0.548	0.643	0.762	0.786	0.275	0.762
`pydata/xarray`	7	0.929	1.000	0.571	0.500	0.307	0.857
`pytest-dev/pytest`	6	0.708	1.000	0.843	1.000	0.722	0.833
`psf/requests`	3	0.444	0.667	1.000	1.000	0.399	1.000
`pylint-dev/pylint`	3	0.333	0.333	0.083	0.167	0.067	0.167
`mwaskom/seaborn`	2	0.750	1.000	1.000	0.750	0.667	1.000
`pallets/flask`	1	1.000	1.000	1.000	1.000	0.143	1.000

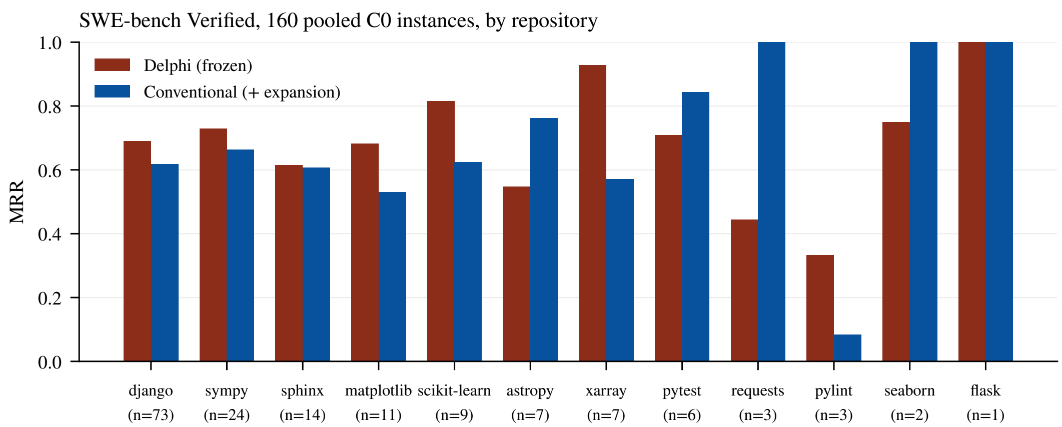

Figure 9: Per-repository MRR on the 160 pooled SWE-bench instances, Delphi against the final conventional rung. Delphi leads on the two largest clusters (`django`, `sympy`) and trails on several small ones; the repository-cluster bootstrap in Table 1 is what turns this heterogeneity into the reported intervals.

Table 10: ARB round-3 partition (C2) by workflow. The concentration of Delphi's lead in `trace2code` and `edit2ripple` is the hypothesis discussed in §4; the partition is validation evidence.

Workflow	n	Delphi		Conventional (+ expansion)		Lexical ranker	
		MRR	R@20	MRR	R@20	MRR	R@20
`trace2code`	38	0.579	0.895	0.149	0.368	0.177	0.711
`edit2ripple`	44	0.383	0.650	0.208	0.642	0.250	0.616
`comment2context`	55	0.303	0.567	0.260	0.691	0.172	0.542
`code2test`	83	0.211	0.588	0.257	0.610	0.055	0.255

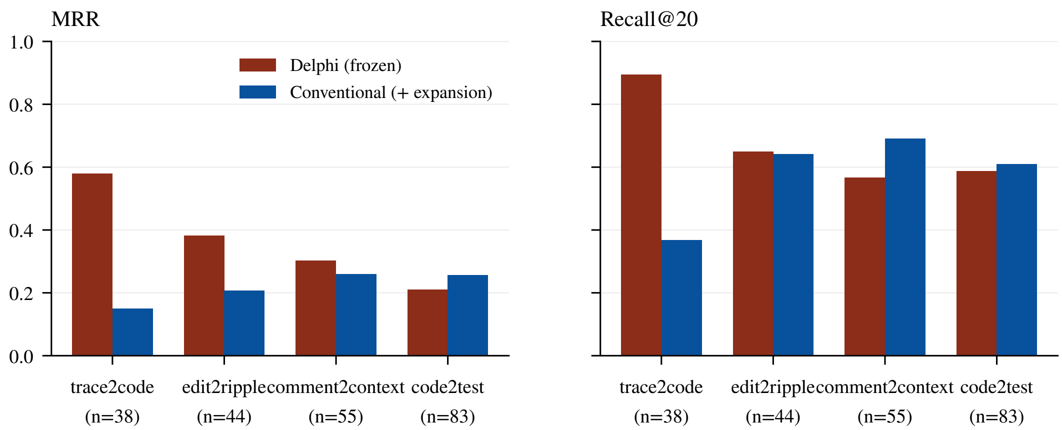

Figure 10: ARB round-3 partition (C2) by workflow, Delphi against the final conventional rung. Queries in `trace2code` carry stack traces with file paths and in `edit2ripple` carry diffs with paths and symbols; `comment2context` and `code2test` carry prose or code without paths.

Table 11: Median query latency in seconds per system and set (wall clock on a warm process, index construction excluded, hosted API round trips included). The lexical comparators' round-3 SWE-bench latencies were recorded in round 3 and are about five times their expansion-set values on the same repositories; the ranker code is identical, so the difference is harness conditions, and we do not compare those two columns.

System	Independent	SWE-bench r3	SWE-bench exp.	ARB (C2)
BM25	1.23	10.66	2.19	1.27
Lexical ranker	1.11	11.03	2.20	1.37
Lexical+BM25 RRF	1.57	13.22	4.19	1.93
Dense	0.49	0.51	0.75	0.50
Dense+BM25 RRF	0.20	2.02	2.04	0.53
+ Delphi's rerankers	2.25	3.72	3.78	2.14
+ expansion	4.06	6.20	6.18	4.22
Delphi (frozen)	3.67	3.63	6.36	4.31

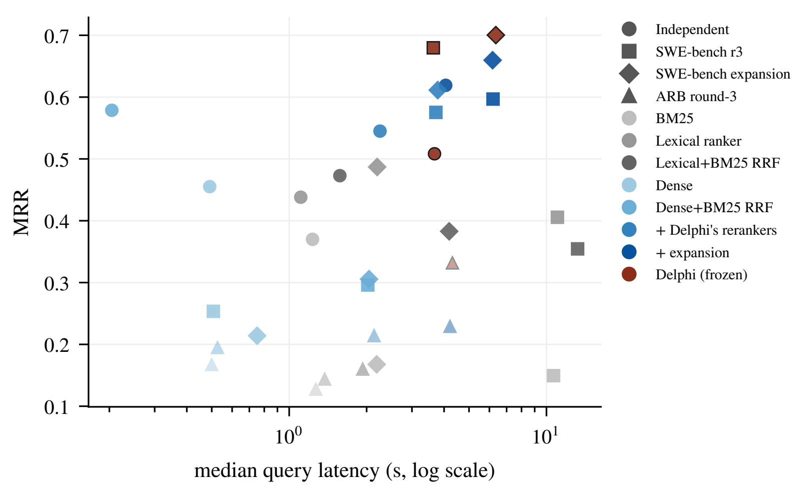

Figure 11: Median query latency against MRR for every system and set (marker shape: set; colour: system; ARB shown faded). The two hosted-LLM rungs and Delphi occupy the same latency band, 2–6 seconds per query; the dense rung is an order of magnitude faster and, on the independent set, competitive.

E DEVELOPMENT ABLATIONS AND NEGATIVE RESULTS

Round-3 preregistered ablations (C1 development sets). Exact vector scan replaced HNSW: $+0.020$ Recall@20 $[0.000, 0.063]$ on the ARB development set, with HNSW additionally implicated in run-to-run variance (Appendix G); accepted. Query expansion off versus on: expansion stayed on. Gemini embedding swap (`gemini-embedding` in place of `text-embedding-3-small`): MRR -0.054 $[-0.102, -0.011]$; rejected. File-level Okapi BM25 branch (whole-file BM25 as a seventh fused branch): the single approved configuration failed three preregistered gates (MRR interval floor -0.050 and BCY floor -0.037 against a -0.01 gate on each; warm median latency ratio 1.34 against a 1.20 gate) despite a non-negative Recall@20 delta and zero any-gold losses; rejected and shipped default-off. Two development findings moved code: review-comment queries retrieved the file the comment already quoted, so paths present in the query envelope are demoted after the cross-encoder (quoted-path demotion); and a global `*.pb.go` exclusion had silently dropped a gold file, so agent-mode indexing now retains generated source under the existing size and binary guards.

Round-2 components, accepted on the ARB v2 pool (now C2). Aligning the query-time embedding model with the index moved workflow-macro MRR from 0.055 to 0.173; reciprocal-rank fusion replaced max-normalized score fusion; the path-affinity branch made file paths searchable; the `ms-marco-MiniLM` cross-encoder at $k{=}30$, $\alpha{=}0.4$ moved MRR $0.176 \rightarrow 0.193$; hypothetical-document expansion moved held-out MRR $0.228 \rightarrow 0.241$ and Recall@20 $0.552 \rightarrow 0.579$; listwise reranking moved held-out MRR $0.241 \rightarrow 0.285$, Recall@5 $0.355 \rightarrow 0.419$, and BCY@8k $0.376 \rightarrow 0.446$. These are the decisions that make the round-3 ARB partition C2.

Round-2 ideas measured and rejected. Recorded so that the negative space of the design is visible. Four of these looked like wins on the 75-case development split and lost on the 220-case held-out split; development differences under about 0.03 were not trustworthy at that size, and run-to-run variance with an LLM in the loop was about 0.02.

- *File-evidence aggregation* (best chunk plus damped support from other chunks): Recall@20 $0.544 \rightarrow 0.467$ at $w{=}0.5$; many gold files match on exactly one chunk.
- *Deeper candidate pools* (50, 100, 150 per branch): MRR $0.197/0.195/0.194$; candidate supply was not the constraint.
- *Deeper rerank window* ($k{=}30, 60, 100$): Recall@20 $0.560/0.552/0.444$; the cross-encoder promotes plausible files from the tail.
- *Pure cross-encoder ordering* ($\alpha{=}1.0$): MRR $0.193 \rightarrow 0.154$, the worst configuration tested.
- *Larger code-aware reranker* (`bge-reranker-base`, 278M): lost on every metric and ran $7\times$ slower (21.3 s vs. 2.9 s).
- *Fusion weight rebalancing* (five configurations): defaults at or near the optimum; lexical-heavy clearly worse once embeddings were aligned.
- *Reverse-dependency branch* at a global weight: `edit2ripple` Recall@5 $0.238 \rightarrow 0.381$ but overall MRR $0.193 \rightarrow 0.163$; gated on query intent, the harm and the gain both vanished (held-out MRR $+0.004$, Recall@5 -0.007) because the listwise stage already promotes dependents.
- *File path prepended to rerank passages*: development Recall@5 $0.327 \rightarrow 0.333$, held-out $0.355 \rightarrow 0.346$.
- *Wider listwise pool* (20 to 40 candidates): $+0.016$ Recall@20 for -0.016 MRR and -0.008 Recall@5 and $+1.2$ s.
- *Cascade listwise* (second pass over the top 6 with 1,200-character excerpts): $+0.001$ MRR for -0.036 Recall@5 and -0.017 Recall@20.
- *Listwise window and excerpt tuning* ($k{=}12$, 700 characters): development Recall@5 0.411 vs. 0.391, held-out 0.378 vs. 0.419.

The diagnostic that these attempts shared: on the held-out split 70.5% of cases had a gold file in the top 20 and 54.1% in the top 5, so the remaining gap was inside the reranker's window, and no change to coverage, fusion, pool size, or prompt shape moved it. The round-4 ladder makes the same point from the other direction: the reranking head is where the margin is, and it is not Delphi's.

F DOCUMENTATION TRACK

Cases. Forty DS-1000 problems (development subset, C1), eight each from NumPy, pandas, Matplotlib, scikit-learn, and TensorFlow, each annotated with a gold API identifier that a correct answer must name. Identifier hit is a case-insensitive substring match of the gold identifier against the scored text.

Arms. *Documentation engine* (hosted): library resolution followed by documentation retrieval; two modes were recorded, "oracle" (the library ID chosen by us) and "quality-guided" (the engine's own top-ranked library), both compacted to the 8,000-token budget. *Delphi*: the frozen stack indexing the same libraries' documentation sources, k=5 and k=20, compacted to the same budget. *Synthesis engine* (hosted): its native answer mode, recorded once per case with its five cited source chunks; its raw-retrieval mode returned no sources during the recorded round. *Reconstructed synthesis-engine retrieval*: the five source chunks extracted from each recorded response (mean 2,754 tokens per case), packed with the identical routine and budget. *Frozen synthesis stage*: `gpt-5.4-mini`, temperature 0, 1,600 output tokens, the system prompt below, three repeats per arm. *Control*: the same model and repeats with the control prompt and no retrieved context.

Synthesis system prompt (verbatim).

```
You are a documentation-grounded coding assistant.
Answer the coding question directly and concretely.
Name the exact library APIs, functions, methods, and
parameters that solve the task, using their precise
identifiers.
Ground your answer in the retrieved documentation when it is
relevant, and cite the source paths you used.
If the retrieved documentation is missing or unhelpful,
still answer from your own knowledge of the library and say
the documentation did not cover it.
Do not use Markdown code fences.
```

The user turn is <coding_question>...</coding_question> followed by <retrieved_documentation>...</retrieved_documentation>.

Control system prompt (verbatim).

```
You are a coding assistant.
Answer the coding question directly and concretely.
Name the exact library APIs, functions, methods, and
parameters that solve the task, using their precise
identifiers.
Answer from your own knowledge of the library.
Do not use Markdown code fences.
```

Table 12: Matched-output-contract documentation comparison, 40 development cases (C1) $\times$ 3 repeats through one frozen synthesis stage. Deltas are case-cluster bootstrap 95% intervals against the no-retrieval control. The synthesis engine's native single pass is shown for reference only.

Arm	Identifier hit	Per-repeat	Δ vs. control
Documentation engine retrieval + frozen synthesis	0.625	0.650/0.600/0.625	+0.133 [+0.008, +0.258]
Delphi retrieval + frozen synthesis	0.617	0.600/0.625/0.625	+0.125 [+0.000, +0.250]
Synthesis engine retrieval + frozen synthesis	0.583	0.625/0.525/0.600	+0.092 [−0.008, +0.200]
Synthesis engine, native synthesis (recorded)	0.575	single pass	—
Model alone (no retrieval)	0.492	0.475/0.475/0.525	—

Raw retrieval. Identifier hit on the raw retrieved context: Delphi k=5 0.400, k=20 0.425; documentation engine 0.375 in both the quality-guided and the oracle-ID mode; reconstructed synthesis-engine sources 0.475. The synthesis engine's raw retrieval is therefore ahead of Delphi by 0.05–0.075 on this set, a difference the frozen synthesis stage then erases (Table 12).

Table 13: Engine-versus-engine paired differences under the matched contract (case-cluster bootstrap 95% intervals; wins/losses/ties over 40 cases). Every interval includes zero.

Comparison	Δ	95% CI	W/L/T
Delphi − synthesis engine (matched)	+0.033	[−0.092, +0.158]	7/5/28
Delphi − synthesis engine (native)	+0.042	[−0.092, +0.183]	6/6/28
Delphi − documentation engine	−0.008	[−0.117, +0.092]	6/5/29
Documentation engine − synthesis engine (matched)	+0.042	[−0.083, +0.167]	10/7/23
Synthesis engine matched − native	+0.008	[−0.050, +0.075]	5/4/31

DS-1000 execution. A separate arm executes generated code against the DS-1000 tests with a frozen generator (40 cases $\times$ 3 repeats): no retrieval 0.575; documentation engine 0.567; Delphi 0.592; synthesis engine 0.642. Every paired interval against the control spans roughly ± 0.13 (§7).

G Determinism

Probes. Eight ARB development queries repeated ten times each against the July stack (mean exact top-10 rate 30.6%), with paired traces of every stage. Three sources were isolated: the listwise and expansion stages were uncached and unseeded; the fusion SQL broke ties by insertion order; and HNSW returned different neighbour sets under concurrent load. Fixes: total ordering in every SQL stage (score, then path, then chunk ID), single-flight sharing of concurrent identical hosted calls, an explicit LLM seed, and a persistent SQLite stage cache keyed by model, prompt, and seed. After the fixes: 100% exact repeats in-process across two concurrent 8×10 packets and two 75-case repeats; a cold restart moved $7/8$ lists until the cache was warm, after which $8/8$ survived two restarts.

Embedding stability. `text-embedding-3-small`: N=20 repeats of the same texts differed at the fifth decimal and never changed top-ten membership; Gemini embeddings were bitwise identical across repeats.

Like-for-like measures (Table 2, Figure 12). Ten documentation cases $\times$ ten runs per engine, all $\binom{10}{2} \times 10 = 450$ run pairs. *Retrieved set exact*: the set of retrieved units is identical (units: retrieved items for Delphi and the documentation engine; the set of cited URLs for the synthesis engine, whose retrieval is otherwise unobservable). *Retrieved order exact*: the ordered list is identical. *Context byte-exact*: the delivered text is identical. *Identifier-hit agreement*: the scored outcome agrees across the pair. The earlier report compared Delphi's and the documentation engine's byte-exact context rate with the synthesis engine's byte-exact *answer* rate; that is the comparison Table 2 replaces.

H Executable pilot

Protocol (preregistered before any run). Tasks: the 62 round-3 SWE-bench Verified instances (C0). Agent: mini-SWE-agent 2.4.6, text-based action format, `gpt-5.4-mini`, official `x86_64` instance images run under emulation on an Apple-silicon host with `--platform linux/amd64`. Conditions: `none`; `random` (five candidate files drawn with seed 1042 per instance, gold excluded); `delphi` (top five of the frozen stack's recorded ranking); `hybrid_rerank_expand` (top five of the conventional ladder's recorded ranking). Seed block: identical wording, up to five paths with the first 40 lines of each file at the base commit, capped at 3,000 tokens. Budgets: step limit 50 with a 2.0 USD cost cap; a tighter secondary budget was preregistered and not run because agents used 7–8 steps of 50. Outcome: verified repair under `swebench` 5.0.2 against `SWE-bench/SWE-bench_Verified`; secondary outcomes API cost, steps, and submission rate. Analysis: paired by instance; instance-cluster and repository-cluster bootstrap 95% intervals (20,000 resamples, seed 1042); exact McNemar on discordant pairs. An interval including zero at n=62 is reported as unresolved. Repeats: two independent trajectories per instance and condition (496 trajectories), pooled as per-instance means.

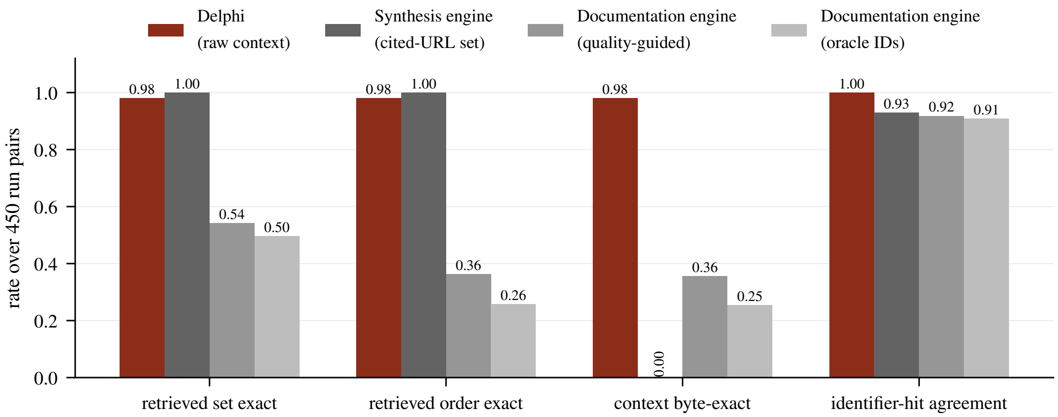

Figure 12: Like-for-like repeatability over 450 run pairs per engine (Table 2). The synthesis engine's cited-source set never changes while its answer text always does; Delphi's retrieved text is byte-exact in 98% of pairs and its scored outcome in 100%.

Seed block (verbatim shape).

```
<retrieved_context>
A code search engine retrieved the following repository
files as potentially relevant to this issue, best match
first.  Treat them as hints and verify them with your own
tools; the fix may involve other files.
### path/to/first_file.py (lines 1-40)
...first 40 lines of the file at the base commit ...
### path/to/second_file.py (lines 1-40)
...
</retrieved_context>
```

When the 3,000-token budget would be exceeded, a file's head is dropped and only its path is listed; when even the path does not fit, the block ends. The random condition uses the same block with five files drawn uniformly from the snapshot's non-gold files.

Table 14: Pooled pilot results: per-instance means over the two repeats. * instance-cluster interval excludes zero.

Condition	Resolved (mean of 2 repeats, of 62)	Rate	Mean cost (USD)	Mean steps
No seed	25.0	0.403	0.0190	8.0
Random files (control)	28.5	0.460	0.0251	8.3
Delphi top-5 (frozen)	28.0	0.452	0.0212	7.2
Conventional ladder top-5	24.0	0.387	0.0232	7.5
Paired differences in per-instance resolved rate: instance-cluster 95% CI [repository-cluster CI]; instances favouring a/b				
Delphi − no seed	+0.048 [−0.008, +0.113] [[+0.017, +0.106]]; 6/2			
Delphi − random	−0.008 [−0.073, +0.056] [[−0.081, +0.022]]; 6/8			
Delphi − conventional	+0.065 [+0.000, +0.137] [[+0.013, +0.147]]; 12/5			
Conventional − no seed	−0.016 [−0.089, +0.056] [[−0.106, +0.062]]; 5/8			
Conventional − random	−0.073* [−0.145, −0.008] [[−0.186, −0.024]]; 3/9			
Random − no seed	+0.056 [−0.016, +0.137] [[+0.005, +0.179]]; 9/5			

Reading the repeats. In the first repeat the Delphi seed resolved 31 instances against 25 for no seed, an apparent +0.097 that a single-repeat analysis would have reported with an instance-cluster interval excluding zero; in the second repeat both resolved 25. The conventional seed resolved 23 in the first repeat and 25 in the second. Pooled over repeats, the only interval excluding zero is the conventional seed's deficit against the random control. Sampling variance across trajectories of the same instance is therefore of the same order as the between-condition differences, and any single-trajectory pilot of this size should be read as inconclusive regardless of its point estimate.

Table 15: First repeat alone.

Condition	Resolved (of 62)	Rate	Mean cost (USD)	Mean steps
No seed	25.0	0.403	0.0162	7.7
Random files (control)	30.0	0.484	0.0263	9.0
Delphi top-5 (frozen)	31.0	0.500	0.0208	7.3
Conventional ladder top-5	23.0	0.371	0.0240	7.5
Paired differences in per-instance resolved rate: instance-cluster 95% CI [repository-cluster CI]; instances favouring a/b				
Delphi − no seed	+0.097* [+0.016, +0.194] [[+0.019, +0.244]]; 7/1			
Delphi − random	+0.016 [−0.065, +0.097] [[−0.057, +0.154]]; 4/3			
Delphi − conventional	+0.129* [+0.032, +0.242] [[+0.029, +0.317]]; 10/2			
Conventional − no seed	−0.032 [−0.113, +0.048] [[−0.147, +0.045]]; 2/4			
Conventional − random	−0.113* [−0.210, −0.032] [[−0.208, −0.059]]; 1/8			
Random − no seed	+0.081 [−0.016, +0.177] [[−0.015, +0.200]]; 7/2			

Table 16: Second repeat alone.

Condition	Resolved (of 62)	Rate	Mean cost (USD)	Mean steps
No seed	25.0	0.403	0.0219	8.3
Random files (control)	27.0	0.435	0.0238	7.6
Delphi top-5 (frozen)	25.0	0.403	0.0216	7.2
Conventional ladder top-5	25.0	0.403	0.0224	7.5
Paired differences in per-instance resolved rate: instance-cluster 95% CI [repository-cluster CI]; instances favouring a/b				
Delphi − no seed	+0.000 [−0.097, +0.097] [[−0.097, +0.061]]; 5/5			
Delphi − random	−0.032 [−0.129, +0.065] [[−0.219, +0.040]]; 4/6			
Delphi − conventional	+0.000 [−0.097, +0.097] [[−0.074, +0.027]]; 4/4			
Conventional − no seed	+0.000 [−0.097, +0.097] [[−0.082, +0.094]]; 5/5			
Conventional − random	−0.032 [−0.129, +0.048] [[−0.189, +0.023]]; 3/5			
Random − no seed	+0.032 [−0.065, +0.129] [[−0.034, +0.194]]; 6/4			

Seed quality. Delphi's top five named at least one gold file on 47 of 62 instances; the conventional ladder's on 42; the random block by construction on none. Agents in every condition retained ordinary shell tools and could, and did, locate files the seed had not named.

Cost and steps. Mean cost per trajectory: no seed \$0.019, random \$0.025, Delphi \$0.021, conventional \$0.023; mean steps 8.0, 8.3, 7.2, 7.5 respectively. Total spend for all 496 trajectories was \$10.97. All but two trajectories terminated by submission; the two exceptions (random condition, first repeat) hit the step or cost limit and count as unresolved.

I Hosted engine accounting

Documentation engine. Accessed through its public REST API (v2) with a project credential. Two retrieval modes were recorded (oracle library ID and quality-guided); the quality-guided mode is the fair comparison because it does not receive the gold library. Its retrieved documentation drifted across runs (54% retrieved-set exact), which is the engine's behaviour and not an artifact of our client.

Synthesis engine, documentation surface. Accessed through its hosted API. The native answer mode was recorded once per case in round 3 with its cited sources; the raw-retrieval mode returned no sources during that round. Round 4 reconstructed the retrieved context from the recorded responses rather than issuing new calls, so the matched arm reflects the engine's round-3 retrieval exactly. Determinism probes (ten cases $\times$ ten runs) were issued in round 3.

Synthesis engine, repository surface. Indexing the ARB development split requires 68 exact `(repo, base_commit)` snapshots. Round 2 indexed 32 pairs through a sharded upload; whole-snapshot ingestion of the large repositories entered a `syncing` state that never reached a terminal status. Round 3 issued a minimal one-shard probe that remained `syncing` across a 40-minute deadline and a three-hour watch. On 2026-09-09 a fresh one-shard probe under a newly provided credential (which the account inventory shows resolves to the same account, with the August probes still `syncing`) again timed out after 40 minutes. No repository query was ever scored against the engine, so no number is reported in either direction. We do not attribute the stall to the engine's design; it may be an account, quota, or ingestion-path issue that a different account would not encounter.

Credentials. Hosted runs used project credentials supplied by the authors; none of the recorded artifacts contain them, and re-running the hosted arms requires the reader's own.

J COMPUTE AND COST

All round-4 experiments ran on 2026-09-09 on one Apple-silicon workstation with a local PostgreSQL instance, using hosted APIs for embeddings and LLM stages. The matched ladder embedded 233.5M new tokens across its runs (3,807 embedding requests, roughly $4.70 at list price); its persistent embedding cache holds 1,440,990 vectors (11.9 GB of SQLite), and its LLM stage cache holds 1,051 listwise and expansion replies, so any rung can be re-executed bitwise without hosted calls. Delphi's own stage cache for the frozen stack is archived separately. The executable pilot ran 496 trajectories (4 conditions $\times$ 2 repeats $\times$ 62 instances) with four concurrent Docker workers under `amd64` emulation for a total API spend of $10.97; harness evaluation ran the official SWE-bench images once per trajectory. The documentation matched arm reused recorded hosted responses and issued 120 frozen-synthesis calls. Round-3 costs (the Delphi runs, lexical comparators, determinism probes, and hosted documentation calls) are recorded in the archive's journals.

K ARTIFACT RELEASE

Everything below is contained in a single archive and will be released publicly, together with the frozen Delphi source at commit `91d76c1`, once the de-identification pass is complete; an anonymized copy accompanies the submission and the public repository URL is withheld for review.

- `results/`: 487 files (221 MB). Per-run `*-details.jsonl` (one line per case: query, gold paths, ranked paths, per-case metrics, latency, served configuration) and `*-summary.json` for every system on every set, including development and rejected configurations; every paired-analysis JSON with bootstrap intervals and McNemar counts; the like-for-like determinism analysis; the documentation synthesis outputs for every arm and repeat; the exposure ledger; the expansion index audit; the ladder's embedding and LLM caches.
- `results/pilot/`: the four seeded datasets, all 496 mini-SWE-agent trajectories (`*.traj.json` with every model call and action), the official harness evaluation reports and logs per condition and repeat, and the three analysis files.
- `samples/`, `samples-ds1000/`: split manifests and case files for every set, including the expansion draw manifest with the two preprocessing rules applied.
- `harness/`: the runner for every static system (Delphi, the matched ladder in `strong_baselines.py`, the lexical comparators), the paired-analysis scripts, the exposure-ledger builder, the corpus restorer (bare clones at recorded commits), the expansion sampler, the pilot dataset builder and analyzer, and `paper_tables.py` / `paper_figures.py`, which regenerate every table and figure in this paper.
- `ds1000_harness/`: the documentation-track runners, the synthesis stage, and the synthesis-engine context reconstruction.
- `stack/`: the frozen worktree, the native launch script (Appendix A), the pilot run script, and the compose files of the round-3 ablations.
- `context/`: the journals (state, decisions, hypotheses, workstreams), the preregistered pilot protocol, the artifact inventory with SHA-256 of every table's inputs, and the exposure ledger in Markdown.
- `round2-provenance/`: the round-2 protocol, source and statistics locks, capability audits of the hosted engines, and the measured-and-rejected record (Appendix E).

Repository corpora (58 repositories, 365 snapshots) are not redistributed; `harness/restore_corpora.py` re-clones them from public GitHub at the recorded commits. Hosted-engine artifacts are the recorded responses; re-running those arms needs the reader's own credentials.

L CORRECTIONS TO OUR EARLIER REPORT

A preliminary version of this study reported results that this paper corrects. The crosswalk is given so that a reader of the earlier text can see what changed and why.

Earlier statement	Corrected statement	Basis
The 220-case ARB partition is "untouched"; results on it are confirmatory.	The partition is C2: re-drawn from a pool all of whose cases were scored in round 2, whose aggregates gated shipped components. Validation evidence only.	Exposure ledger (Appendix C); round-2 record (Appendix E).
Delphi's margin over lexical baselines shows the value of its hybrid design.	Most of the margin is a reranking head over a dense+BM25 pool; Delphi's own additions are $+0.06$–0.08 on yield metrics at $n{=}160$ (issue queries) and negative at $n{=}18$ (commit-to-files).	Matched ladder (§4, Figure 1).
Delphi 98.0% byte-identical vs. 35.6% and 0.0% for the hosted engines.	Retrieved-set stability: synthesis engine 1.000, Delphi 0.980, documentation engine 0.54. The 0.0% measured generated text.	Like-for-like determinism (§5).
Under a matched contract Delphi reaches "parity or better" with the synthesis engine.	No two engines are statistically distinguishable at $n{=}40$; on raw retrieval the synthesis engine is ahead. Parity is not established.	Four-arm matched comparison (§6).
The oracle ceiling shows retrieval dominates the agent's failures.	Withdrawn. The closed-tool oracle shows localization headroom in that harness only.	§7.
No executable evidence.	Preregistered pilot: no seed effect resolved at $n{=}62 \times 2$; random files do as well as Delphi's.	§7, Appendix H.